

```
PS C:\Users\VICTUS_User\lab10-673380596-5-sec4> curl.exe -i -X GET http://localhost:8080/products
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/json

[{"id":"1","name":"iPhone 15 Pro (673380596-5 SEC 4)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0},{
"id":"2","name":"MacBook Air M3","category":"Electronics","brand":"Apple","stock":20,"price":49900.0,"discountType":"NONE","discountedPrice":49900.0},{
"id":"3","name":"Samsung Galaxy S24","category":"Electronics","brand":"Samsung","stock":30,"price":29900.0,"discountType":"SEASONAL","discountedPrice":23920.0}]
```

ดึงสินค้าทั้งหมด

```
PS C:\Users\VICTUS_User\lab10-673380596-5-sec4> curl.exe -i -X GET http://localhost:8080/products/1
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 171

{"id":"1","name":"iPhone 15 Pro (673380596-5 SEC 4)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0}
```

ดึงสินค้าเดียว

```
PS C:\Users\VICTUS_User\lab10-673380596-5-sec4> curl.exe -i -X DELETE http://localhost:8080/products/2
HTTP/1.1 200 OK
content-length: 0
```

ลบสินค้าตาม id

```
PS C:\Users\VICTUS_User\lab10-673380596-5-sec4> curl.exe -i -X GET http://localhost:8080/products/category/Electronics
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/json

[{"id":"1","name":"iPhone 15 Pro (673380596-5 SEC 4)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0},{
"id":"3","name":"Samsung Galaxy S24","category":"Electronics","brand":"Samsung","stock":30,"price":29900.0,"discountType":"SEASONAL","discountedPrice":23920.0}]
```

ดึงสินค้าตาม category

```
PS C:\Users\VICTUS_User\lab10-673380596-5-sec4> curl.exe -i -X GET http://localhost:8080/products/1/price
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 7

35910.0
```

ดึงราคาหลังส่วนลดของสินค้า

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Reactive Programming: ใช้ non-blocking I/O และ asynchronous stream ซึ่ง thread ไม่ต้องรอ สามารถจัดการหลายงานพร้อมกันได้ โดยใช้ Mono, Flux ส่งข้อมูลเมื่อพร้อม เหมาะกับระบบที่มีผู้ใช้เยอะ ๆ

Blocking: เรียก DB หรือ API แล้วจะต้องรอผลลัพธ์ก่อนถึงจะทำงานต่อ ทำให้ thread ถูกบล็อก และไม่สามารถทำงานอื่นได้ ถ้า request เยอะๆ จะส่งผลให้ช้า

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono: ใช้สำหรับผลลัพธ์ **1 ค่า** หรือ **ไม่มีค่าเลย** (เช่น findById, deleteById, getPrice)

Flux: ใช้สำหรับผลลัพธ์ **หลายค่า** (stream/list) เช่น findAll, findByCategory

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

```
public class ProductWebClient {

    private final WebClient client = WebClient.create("http://localhost:8080");

    public Mono<Product> getProductById(String id) {
        return client.get()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class);
    }
}
```

```
Mono<Product> productMono = client.get()
```

```
// เริ่มต้นด้วย HTTP GET
```

```
.uri("/products/1")
```

```
// ระบุ URI
```

```
.retrieve()
```

```
// ส่ง request และเตรียมรับ response
```

```
.bodyToMono(Product.class);
```

```
// แปลง response body เป็น Mono<Product>
```

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

```

package com.example.lab10.controller;

import com.example.lab10.model.Product;

import com.example.lab10.service.ProductService;

import org.springframework.web.bind.annotation.*;

import reactor.core.publisher.Flux;

import reactor.core.publisher.Mono;

@RestController

@RequestMapping("/products")

public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {

        this.service = service;

    }

    // GET /products -> คืน Flux<Product>

    @GetMapping

    public Flux<Product> getAll() {

        return service.getAll();

    }

    // GET /products/{id} -> คืน Mono<Product>

    @GetMapping("/{id}")

    public Mono<Product> getById(@PathVariable String id) {

        return service.getById(id);
    }
}

```

```

    }

    // POST /products -> คืน Mono<Product>

    @PostMapping

    public Mono<Product> save(@RequestBody Product product) {

        return service.save(product);

    }

    // DELETE /products/{id} -> คืน Mono<Void>

    @DeleteMapping("/{id}")

    public Mono<Void> delete(@PathVariable String id) {

        return service.delete(id);

    }

    // GET /products/category/{cat} -> คืน Flux<Product>

    @GetMapping("/category/{cat}")

    public Flux<Product> getByCategory(@PathVariable String cat) {

        return service.getByCategory(cat);

    }

    // GET /products/{id}/price -> คืน Mono<Double>

    @GetMapping("/{id}/price")

    public Mono<Double> getDiscountedPrice(@PathVariable String id) {

        return service.getDiscountedPrice(id);

    }

}

```

```
package com.example.lab10.service;

import com.example.lab10.model.Product;

import com.example.lab10.repository.ProductRepository;

import org.springframework.stereotype.Service;

import reactor.core.publisher.Flux;

import reactor.core.publisher.Mono;

@Service

public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {

        this.repository = repository;

    }

    // ดึงสินค้าทั้งหมด

    public Flux<Product> getAll() {

        return repository.findAll();

    }

    // ดึงสินค้าตาม id

    public Mono<Product> getById(String id) {

        return repository.findById(id);

    }

    // บันทึกสินค้าใหม่

    public Mono<Product> save(Product product) {

        return repository.save(product);

    }

}
```

```

    }

    // ลบสินค้า

    public Mono<Void> delete(String id) {

        return repository.deleteById(id);

    }

    // ดึงสินค้าตาม category

    public Flux<Product> getByCategory(String category) {

        return repository.findByCategory(category);

    }

    // คำนวณราคาหลังส่วนลด

    public Mono<Double> getDiscountedPrice(String id) {

        return repository.findById(id)

            .map(Product::calculateDiscountedPrice); // ใช้ map เพื่อแปลง
Product -> Double

    }

}

```

```

package com.example.lab10.repository;

import java.util.Map;

import java.util.concurrent.ConcurrentHashMap;

import com.example.lab10.model.Product;

import reactor.core.publisher.Flux;

import reactor.core.publisher.Mono;

public class ProductRepository {

    private final Map<String, Product> store = new ConcurrentHashMap<>();

    public ProductRepository() {

        store.put("1", new Product("1", "iPhone 15 Pro (673380596-5 SEC
4)",

            "Electronics", "Apple", 50, 39900.0, "MEMBER"));

        store.put("2", new Product("2", "MacBook Air M3",

            "Electronics", "Apple", 20, 49900.0, "NONE"));

        store.put("3", new Product("3", "Samsung Galaxy S24",

            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));

    }

    // 1. หา Product 1 รายการ

    public Mono<Product> findById(String id) {

        Product product = store.get(id);

        return product != null ? Mono.just(product) : Mono.empty();

    }

    // 2. หา Product ทั้งหมด

```

```
public Flux<Product> findAll() {  
  
    return Flux.fromIterable(store.values());  
  
}  
  
// 3. บันทึก Product  
  
public Mono<Product> save(Product product) {  
  
    store.put(product.getId(), product);  
  
    return Mono.just(product);  
  
}  
  
// 4. ลบ Product  
  
public Mono<Void> deleteById(String id) {  
  
    store.remove(id);  
  
    return Mono.empty();  
  
}  
  
// 5. กรองตาม category  
  
public Flux<Product> findByCategory(String category) {  
  
    return findAll()  
  
        .filter(p -> p.getCategory().equalsIgnoreCase(category));  
  
}  
  
}
```